쿠폰 토큰 역직렬화 데모 분석

OWASP A08 (Software and Data Integrity Failures) 실습 코드 리뷰 · HMAC 서명 이해하기 · 2026년 10월 17일

> 이 문서는 `com.basic.myspringboot.vuln.deserialization` 패키지(7개 파일)에 대해 나눈 질문과 답변을 정리한 것입니다. 코드 분석, HMAC 서명의 발급·검증 흐름, 세 가지 적용 방식 비교, 그리고 서명 검증 메서드를 한 줄씩 따라가는 단계별 설명으로 구성되어 있습니다.

1. 코드 분석
2. HMAC 서명 흐름 이해하기
3. 세 가지 적용 방식 비교
4. 서명 검증 단계별 따라가기
5. 핵심 정리

1. 코드 분석

OWASP Top 10의 A08을 실습하는 강의용 데모입니다. 쿠폰을 Java 직렬화해서 클라이언트에 맡겼다가 다시 받는 구조이고, 같은 문제를 세 가지 방식으로 비교하도록 짜여 있습니다.

1.1 전체 구조

파일	역할
`Voucher`	직렬화 대상 (code, discountPercent, issuedToUuid)
`IssueRequest` / `IssueResponse`	발급 요청(code)과 응답(토큰 포함)
`ApplyRequest` / `ApplyResponse`	적용 요청(토큰, 주문금액)과 결과
`DeserializationDemoController`	5개 엔드포인트
`DeserializationDemoService`	핵심 로직 (정책 맵, 직렬화, HMAC)

흐름: `issue`에서 서버 정책(`COUPON_POLICY`)으로 할인율을 정하고, `Voucher`를 만들어 직렬화한 뒤 Base64로 인코딩해 반환합니다. 클라이언트는 이 토큰을 `apply*`로 되돌려 보냅니다.

엔드포인트	동작
`POST /issue`	쿠폰 코드로 voucherToken 발급 (공통)
`POST /apply` 취약	토큰 속 discountPercent를 그대로 신뢰
`POST /apply-safe` 재검증	code로 할인율을 서버에서 재조회
`POST /issue-signed`	HMAC 서명이 붙은 voucherToken 발급
`POST /apply-signed` 서명	서명 검증을 통과한 토큰만 필드를 신뢰

1.2 세 가지 적용 방식

취약 **`/apply`** — 역직렬화한 객체의 `discountPercent`를 그대로 사용합니다. `int`는 직렬화 스트림에 4바이트 빅엔디안으로 그대로 들어가므로, Base64를 디코딩해 그 4바이트만 바꾸면 `WELCOME10` 쿠폰이 100% 할인이 됩니다. 원인은 클라이언트가 들고 있던 데이터를 서버가 발급한 것이라 착각하고 신뢰한 데 있습니다.

재검증 `/apply-safe` — 토큰에서 `code`만 꺼내고, 할인율은 서버 정책에서 다시 조회합니다. 토큰 안의 할인율을 바꿔도 무의미하므로 무결성 문제는 해결됩니다. 다만 신뢰할 수 없는 바이트를 역직렬화하는 문제는 그대로 남아 있습니다.

서명 `/apply-signed` — 토큰 형식은 `base64(직렬화바이트).base64(HMAC-SHA256(직렬화바이트))`입니다. 서명을 먼저 검증한 뒤에만 역직렬화하므로, 변조된 토큰은 `ObjectInputStream`에 들어가지도 않습니다. 서명 비교에 `MessageDigest.isEqual`(상수 시간 비교)을 쓰고, 스레드 안전하지 않은 `Mac`을 호출마다 새로 만드는 점도 올바릅니다. JWT의 HS256과 같은 원리입니다.

1.3 짚어볼 만한 문제점

① /apply-safe는 "무결성"만 안전하고 "역직렬화"는 안전하지 않습니다.

클라이언트 바이트를 여전히 `ObjectInputStream.readObject()`에 넣습니다. 지금은 가젯 체인 라이브러리가 없어 RCE는 일어나지 않지만, 의존성이 추가되면 가젯 체인이 생길 수 있고, 중첩 `HashSet` 같은 페이로드로 CPU·메모리를 소진시키는 DoS도 가능합니다. `instanceof Voucher` 검사는 `readObject()`가 끝난 뒤라 이미 늦습니다. `ObjectInputFilter`(JEP 290)로 허용 목록을 두는 것을 권장합니다.

```
ois.setObjectInputFilter(ObjectInputFilter.Config.createFilter(

"com.basic.myspringboot.vuln.deserialization.Voucher;java.lang.String;maxdepth=3;maxbytes=1024;!*"));
```

② code가 null인 토큰을 보내면 NPE가 나서 500 에러가 됩니다.

`COUPON_POLICY`는 `Map.of(...)`로 만든 불변 맵이라 `getOrDefault(null, 0)`에서 `NullPointerException`을 던집니다. 발급 시에는 `@NotBlank`가 있지만 직접 조작한 토큰은 그 검증을 거치지 않습니다. "역직렬화된 객체는 생성자나 검증을 거치지 않는다"는 좋은 교육 포인트입니다.

③ Voucher를 record로 바꾸면 검증 지점을 만들 수 있습니다.

일반 클래스는 역직렬화할 때 생성자가 호출되지 않습니다(`@NoArgsConstructor`도 쓰이지 않음). record는 역직렬화 시 정식 생성자를 거치므로 compact constructor에 검증 로직을 넣을 수 있습니다.

④ 서명 토큰에 만료와 재사용 방지가 없습니다.

`issuedToUuid`는 랜덤 값일 뿐 사용자와 묶여 있지 않아, VIP50 토큰을 무한히 재사용하거나 남에게 넘길 수 있습니다. 서명은 "변조 여부"만 보장합니다. 실무라면 만료 시각(exp), 1회용 nonce, 사용자 바인딩이 필요합니다.

⑤ 응답 값과 실제 계산이 다를 수 있습니다.

취약 엔드포인트에서 `discountPercent=999`로 변조하면 계산은 100%로 clamp되지만 응답의 `appliedDiscountPercent`에는 999가 그대로 나갑니다.

⑥ 할인 계산에서 오버플로가 날 수 있습니다.

`orderAmount * clamped`는 `orderAmount`가 아주 크면 `long` 범위를 넘습니다. `@Max`를 두거나 `Math.multiplyExact`를 쓰는 편이 좋습니다.

⑦ 예외가 어떤 HTTP 상태로 나가는지 확인이 필요합니다.

변조·형식 오류·Base64 디코딩 실패가 모두 `IllegalArgumentException`입니다. 전역 `@RestControllerAdvice`가 없다면 400이 아니라 500으로 응답합니다.

⑧ 실수로 배포되지 않도록 막아 두는 것이 좋습니다.

`/api/vuln/**`가 `permitAll`이므로 `@Profile("lab")` 같은 프로파일로 컨트롤러를 조건부 등록하세요.

`voucher.hmac-secret`도 운영 프로파일에서 반드시 덮어써야 합니다.

1.4 총평

교육 흐름(취약 → 재검증 → 서명)이 명확하고, "서명 먼저 검증 후 역직렬화" 순서와 상수 시간 비교처럼 핵심을 잘 구현했습니다. 보강하면 좋을 점은 두 가지입니다. 첫째, `/apply-safe`가 막는 것은 데이터 조작뿐이고 역직렬화 공격면은 그대로라는 점을 분명히 하세요(①). 둘째, 조작된 객체가 생성자 검증을 우회해 NPE를 일으키는 사례(②)를 추가 실습 포인트로 활용할 수 있습니다.

2. HMAC 서명 흐름 이해하기

HMAC 서명은 두 단계로 나눠 보면 이해하기 쉽습니다. 서버가 토큰을 만들 때 서명을 붙이는 단계, 그리고 돌려받은 토큰의 서명을 확인하는 단계입니다.

2.1 발급 단계 (/issue-signed, serializeSigned())

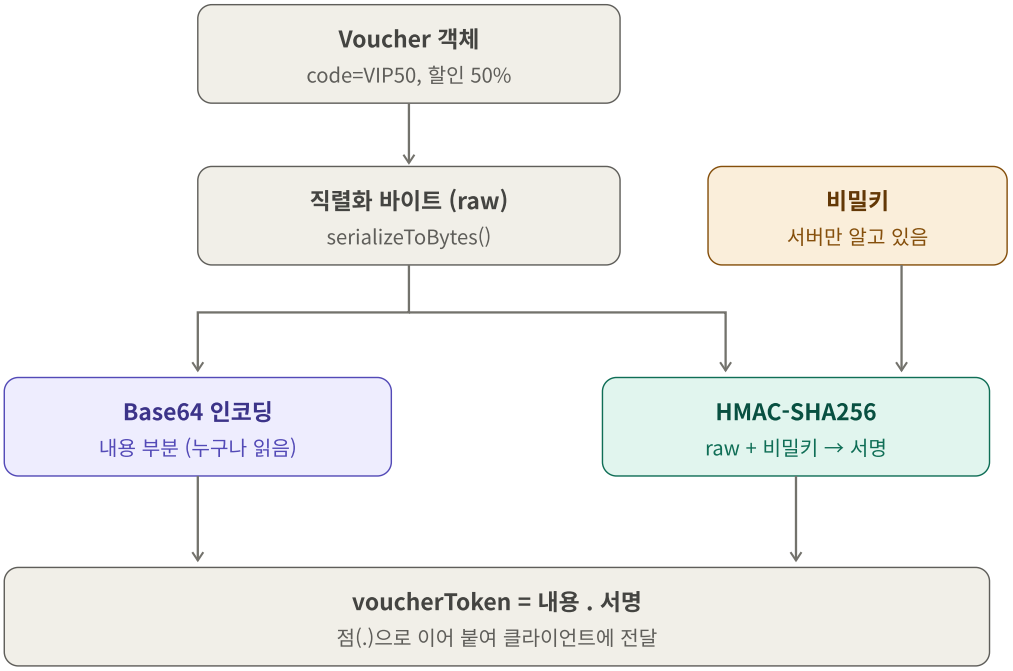


그림 1. 서명이 붙은 토큰의 발급 흐름

서명을 만드는 재료는 두 가지입니다. 하나는 토큰 내용(raw), 다른 하나는 서버만 아는 비밀키입니다. 내용은 Base64일 뿐이라 누구나 읽고 고칠 수 있지만, 서명은 비밀키 없이는 만들 수 없습니다.

2.2 검증 단계 (/apply-signed, deserializeSigned())

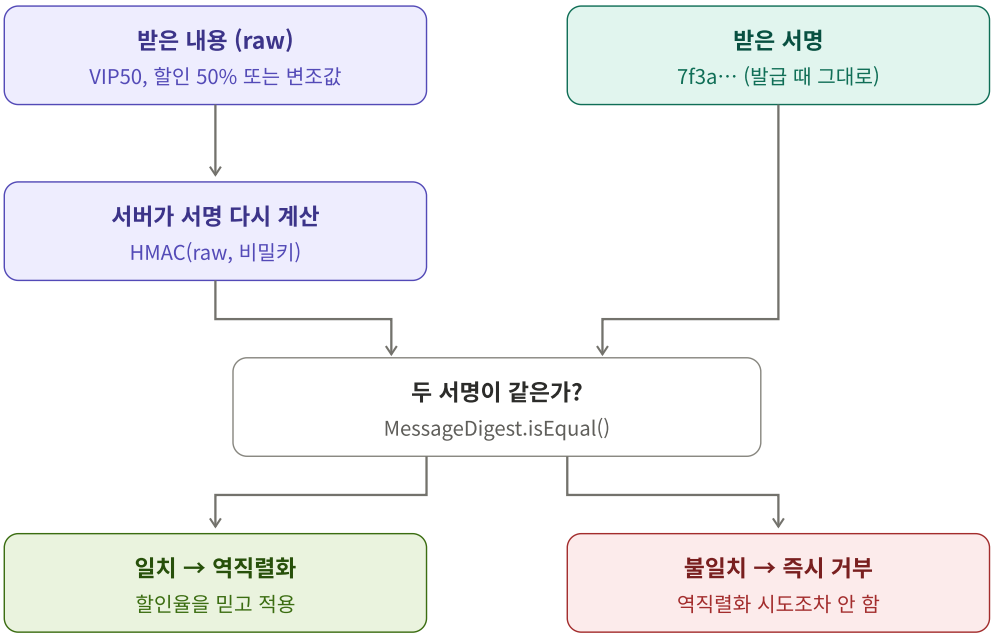


그림 2. 받은 토큰의 서명 검증 흐름 (7f3a…, c91e… 등은 설명용 예시 값)

핵심은 서버가 받은 서명을 "해독"하지 않는다는 점입니다. 서버는 받은 내용으로 서명을 똑같이 다시 만들어 보고, 받은 서명과 같은지만 비교합니다. 정상 토큰이면 다시 계산한 서명도 7f3a…로 같고, 공격자가 할인율을 100%로 바꿨다면 다시 계산한 서명은 c91e…처럼 완전히 다른 값이 되어 거부됩니다.

2.3 비유로 보면

- 토큰 내용은 투명한 봉투에 든 쪽지라서 누구나 읽고 고쳐 쓸 수 있습니다.
- 서명은 서버만 가진 도장(비밀키)으로 찍은 봉인입니다.
- 쪽지를 고치면 봉인 모양이 맞지 않게 되고, 공격자는 도장이 없어서 새 봉인을 찍을 수 없습니다.

2.4 공격자가 시도할 수 있는 방법과 결과

시도	결과
내용만 고치고 서명은 그대로 둠	다시 계산한 서명과 달라서 거부
내용을 고치고 서명도 새로 만듦	비밀키를 몰라서 맞는 서명을 만들 수 없음
정상 토큰을 그대로 다시 사용	통과함. 서명은 재사용을 막지 못하므로 만료 시간 등이 따로 필요

2.5 그림의 각 단계와 코드의 대응

- 내용과 서명을 나누는 부분: `voucherToken.indexOf('.')`에서 `.`을 기준으로 둘로 자릅니다.
- 서명을 다시 계산하는 부분: `byte[] expected = hmac(raw);`
- 비교하는 부분: `MessageDigest.isEqual(expected, signature)` — `Arrays.equals`는 다른 바이트를 만나면 바로 멈추므로 응답 시간 차이로 서명을 한 바이트씩 추측하는 "타이밍 공격"이 가능해집니다.
- 일치할 때만 역직렬화하는 부분: `deserializeFromBytes(raw)`가 검증을 통과한 뒤에 호출됩니다.

이 원리는 JWT와 같습니다. JWT도 `헤더.내용.서명` 구조이고 검증 방식도 똑같습니다.

3. 세 가지 적용 방식 비교

할인율을 50%에서 100%로 바꾼 같은 공격 토큰을 세 엔드포인트에 보냈을 때 각각 어떻게 처리되는지 나란히 비교한 그림입니다.



그림 3. 같은 변조 토큰에 대한 세 엔드포인트의 처리 비교

세 방식의 차이는 "토큰 안의 값을 어떻게 다루느냐" 한 가지로 요약할 수 있습니다.

- `/apply`는 토큰에 적힌 값을 그대로 믿습니다. 클라이언트가 들고 있던 데이터는 언제든 바뀔 수 있으므로 할인율이 그대로 조작됩니다.
- `/apply-safe`는 토큰의 할인율을 아예 쓰지 않고, 쿠폰 코드만 꺼내 서버가 직접 다시 조회합니다. 숫자를 바꿔도 결과는 같습니다. 다만 신뢰할 수 없는 바이트를 역직렬화하는 과정은 남아 있어 null 코드로 인한 에러나 역직렬화 공격 위험이 있습니다.
- `/apply-signed`는 토큰이 발급 당시 그대로인지 서명으로 먼저 확인하므로, 변조된 토큰은 역직렬화 단계까지 가지도 못합니다. 다만 서명은 "내용이 바뀌지 않았다"는 것만 보장하므로 재사용 방지는 따로 필요합니다.

실무에서는 두 방식을 함께 쓰는 경우가 많습니다. 서명으로 변조를 막고, 금액처럼 중요한 값은 서버에서 한 번 더 확인하는 식입니다.

4. 서명 검증 단계별 따라가기

`deserializeSigned()`를 다섯 단계로 나누어, 정상 토큰과 변조 토큰(할인율 50% → 100%)이 각 단계에서 어떻게 다르게 처리되는지 비교합니다. 바이트 값과 토큰 문자열은 이해를 돕기 위한 예시입니다. 실제 서명은 32바이트이고 직렬화 바이트도 훨씬 깁니다. 단, 할인율이 int 4바이트(50이면 `00 00 00 32`)로 들어간다는 점은 실제와 같습니다.

```
private Voucher deserializeSigned(String voucherToken) {
    int dot = voucherToken.indexOf('.');                                    // 1단계
    if (dot < 0) throw new IllegalArgumentException("형식 오류");          // 1단계
    byte[] raw = Base64.getDecoder().decode(voucherToken.substring(0, dot));        // 2단계
    byte[] signature = Base64.getDecoder().decode(voucherToken.substring(dot + 1)); // 2단계
    byte[] expected = hmac(raw);                                            // 3단계
    if (!MessageDigest.isEqual(expected, signature)) {                      // 4단계
        throw new IllegalArgumentException("서명이 유효하지 않습니다");   // 5단계 (변조)
    }
    return deserializeFromBytes(raw);                                       // 5단계 (정상)
}
```

1단계 — 점(.) 위치 찾기

```
    int dot = voucherToken.indexOf('.');
    if (dot < 0) throw new IllegalArgumentException("형식 오류");
```

정상 토큰	변조된 토큰
`rO0ABXNyAC9jb20u…AAAAMg…` . `fzqcEuQFuGE…=`	`rO0ABXNyAC9jb20u…AAAAZA…` . `fzqcEuQFuGE…=`

점의 왼쪽은 내용, 오른쪽은 서명입니다. 점이 없으면 서명이 붙지 않은 토큰이므로 바로 거부합니다. 변조 토큰은 왼쪽 글자 일부가 달라졌지만 오른쪽 서명은 발급 때 그대로입니다.

2단계 — Base64 디코딩

```
    byte[] raw = Base64.getDecoder().decode(voucherToken.substring(0, dot));
    byte[] signature = Base64.getDecoder().decode(voucherToken.substring(dot + 1));
```

정상 토큰	변조된 토큰
raw (노란 칸 = 할인율 4바이트) `ac` `ed` `00` `05` … `00` `00` `00` `32` … → 0x32 = 50% signature (앞 8바이트) `7f` `3a` `9c` `12` `e4` `05` `b8` `61` …	raw (노란 칸 = 할인율 4바이트) `ac` `ed` `00` `05` … `00` `00` `00` `64` … → 0x64 = 100% (변조됨) signature (앞 8바이트) `7f` `3a` `9c` `12` `e4` `05` `b8` `61` …

두 부분을 각각 바이트 배열로 바꿉니다. 이 시점에는 아직 아무것도 믿지 않습니다. 그냥 바이트일 뿐입니다.

3단계 — 서명 다시 계산

```
    byte[] expected = hmac(raw);
```

정상 토큰	변조된 토큰
expected = HMAC(raw, 비밀키) 7f 3a 9c 12 e4 05 b8 61 …	expected = HMAC(raw, 비밀키) c9 1e 44 a0 3b d7 02 8f …

서버는 받은 서명을 "해독"하지 않고, 받은 raw에 자기 비밀키를 섞어 서명을 직접 다시 만듭니다. raw가 1바이트(32 → 64)만 바뀌어도 계산된 서명은 완전히 다른 값이 됩니다. HMAC은 입력이 조금만 달라도 결과가 통째로 바뀌도록 만들어진 함수입니다.

4단계 — 상수 시간 비교

```
    if (!MessageDigest.isEqual(expected, signature)) {
```

정상 토큰	변조된 토큰
expected 7f 3a 9c 12 e4 05 b8 61 signature 7f 3a 9c 12 e4 05 b8 61 **isEqual 결과: true**	expected c9 1e 44 a0 3b d7 02 8f signature 7f 3a 9c 12 e4 05 b8 61 **isEqual 결과: false**

`MessageDigest.isEqual`은 첫 칸부터 마지막 칸까지 모든 바이트를 끝까지 비교합니다. 다른 칸을 만나도 중간에 멈추지 않기 때문에, 응답 시간으로 정답을 추측하는 타이밍 공격이 통하지 않습니다.

5단계 — 결과

정상 토큰	변조된 토큰
`return deserializeFromBytes(raw);` **검증 통과 → Voucher 복원** code=VIP50, discountPercent=50. 발급 당시 그대로임이 보장되므로 믿고 사용합니다.	`throw new IllegalArgumentException(...);` **예외 발생: 서명이 유효하지 않습니다** deserializeFromBytes(raw)는 실행되지 않습니다. 변조된 바이트는 ObjectInputStream에 들어가지도 못합니다.

검증 순서가 핵심입니다. 서명을 먼저 확인하기 때문에, 의심스러운 바이트를 역직렬화하는 위험한 작업을 아예 하지 않습니다. 두 경우는 3단계(서명 재계산)에서 갈리고, 그 뒤의 비교와 결과는 자연스럽게 따라옵니다.

5. 핵심 정리

- 클라이언트에게 맡긴 데이터는 서버가 만든 것이라도 돌아올 때는 "신뢰할 수 없는 입력"입니다.
- 방어 방법은 두 가지입니다. 중요한 값은 서버가 다시 조회하거나(`/apply-safe`), 서명으로 변조 여부를 먼저 확인하는 것(`/apply-signed`)입니다.
- HMAC 서명은 "내용 + 비밀키"로 만들어지며, 서버는 서명을 해독하지 않고 똑같이 다시 만들어 비교합니다.
- 서명 비교는 반드시 상수 시간 비교(`MessageDigest.isEqual`)로 합니다.
- 서명은 변조를 막을 뿐 재사용은 막지 못합니다. 만료 시간, 1회용 처리, 사용자 바인딩이 함께 필요합니다.
- Java 직렬화 자체가 위험 요소이므로, 역직렬화가 꼭 필요하다면 `ObjectInputFilter`로 허용 목록을 두고, 가능하면 JSON + 서명(JWT 등)으로 대체하는 것이 좋습니다.